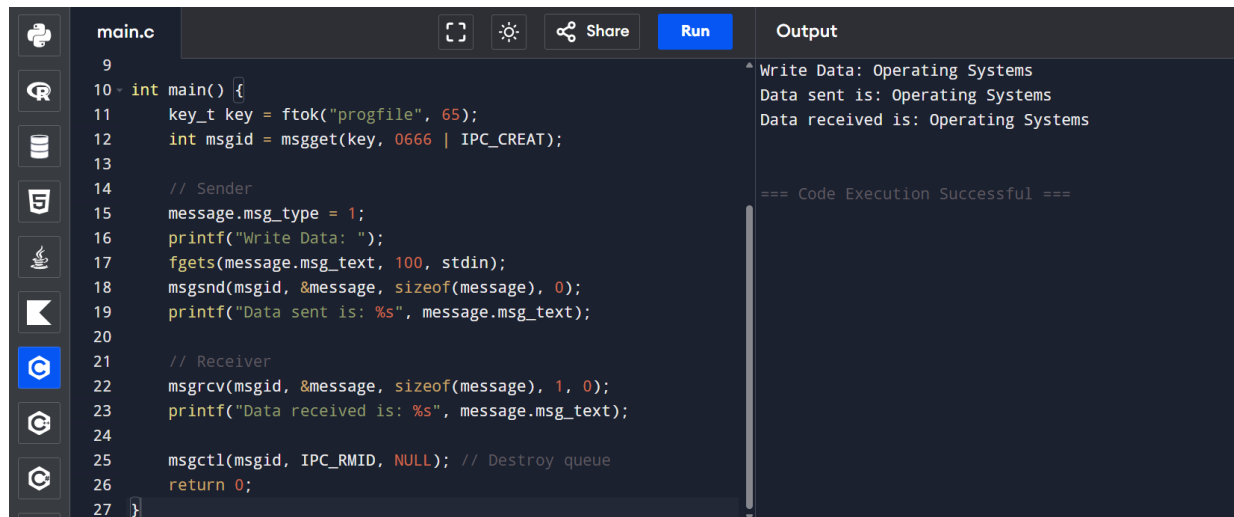


EXPERIMENT 10:

10. Illustrate the concept of inter-process communication using message queue with a C program.

A screenshot of a code editor showing a C program that demonstrates inter-process communication using message queues. The code is in a file named 'main.c'. It defines a 'main' function where a key is generated using 'ftok' and a message queue is created with 'msgget'. The sender part of the code prompts the user to 'Write Data: ', reads the input into 'message.msg_text', and sends it to the queue using 'msgsnd'. The receiver part of the code receives the message using 'msgrcv' and prints it. Finally, the queue is destroyed with 'msgctl' and the program returns 0. The output window on the right shows the execution results: 'Write Data: Operating Systems', 'Data sent is: Operating Systems', and 'Data received is: Operating Systems', followed by '=== Code Execution Successful ==='.

```
9
10 int main() {
11     key_t key = ftok("progfile", 65);
12     int msgid = msgget(key, 0666 | IPC_CREAT);
13
14     // Sender
15     message.msg_type = 1;
16     printf("Write Data: ");
17     fgets(message.msg_text, 100, stdin);
18     msgsnd(msgid, &message, sizeof(message), 0);
19     printf("Data sent is: %s", message.msg_text);
20
21     // Receiver
22     msgrcv(msgid, &message, sizeof(message), 1, 0);
23     printf("Data received is: %s", message.msg_text);
24
25     msgctl(msgid, IPC_RMID, NULL); // Destroy queue
26     return 0;
27 }
```

Output

Write Data: Operating Systems
Data sent is: Operating Systems
Data received is: Operating Systems

=== Code Execution Successful ===

PSEUDO CODE :

1. Define a message structure containing type and text buffer.
2. Generate key using ftok() and create queue with msgget().
3. Sender: Fill structure and send using msgsnd().
4. Receiver: Read message using msgrcv() with matching type.
5. Destroy message queue using msgctl().

CODE:

```
#include <stdio.h>

#include <sys/ipc.h>

#include <sys/msg.h>

struct msg_buffer {

    long msg_type;

    char msg_text[100];

} message;

int main() {

    key_t key = ftok("progfile", 65);

    int msgid = msgget(key, 0666 | IPC_CREAT);

    // Sender

    message.msg_type = 1;

    printf("Write Data: ");

    fgets(message.msg_text, 100, stdin);

    msgsnd(msgid, &message, sizeof(message), 0);

    printf("Data sent is: %s", message.msg_text);

    // Receiver

    msgrcv(msgid, &message, sizeof(message), 1, 0);

    printf("Data received is: %s", message.msg_text);

    msgctl(msgid, IPC_RMID, NULL); // Destroy queue

    return 0;

}
```